

Computers & Security

Understanding Security Weaknesses in Software and Firmware Updates

--Manuscript Draft--

Manuscript Number:	COSE-D-26-01108
Article Type:	VSI:IFIP SEC 2025
Keywords:	Secure Updates Software Updates Firmware Updates Vulnerability Analysis Target-system Identification Mitigation Recommendations
Corresponding Author:	Ahmad B. Usman Linköping University SWEDEN
First Author:	Ahmad B. Usman
Order of Authors:	Ahmad B. Usman Mikael Asplund
Abstract:	This paper presents a study of update-related vulnerabilities based on a curated dataset of disclosures from the past decade. We analyze the exploitability, impact, and severity of these vulnerabilities, comparing them with the broader population of disclosed vulnerabilities. Our findings reveal that update-related vulnerabilities tend to be more severe and impactful, with a higher concentration of high-severity scores and a greater compromise of confidentiality, integrity, and availability. In contrast to general vulnerabilities, which are often network-exploitable, most update-related attacks require local access. We identify and categorize the most common weakness types in update mechanisms. We found that the most frequent weaknesses include authentication and authorization-related weaknesses, input-related weaknesses, path-related weaknesses, and certificate-related weaknesses. Furthermore, we introduce a rule-based approach with predefined keywords that assign vulnerabilities to six target-system classes. The identification relies on textual vulnerability descriptions together with vendor and product metadata. Finally, based on our analysis, we derive a set of clear mitigation recommendations to help secure update processes and reduce associated risks.

Understanding Security Weaknesses in Software and Firmware Updates

Ahmad B. Usman^{a,*}, Mikael Asplund^a

^a*Department of Computer and Information Science, Linköping University, Linköping, Sweden*

ARTICLE INFO

Keywords:

Secure Updates
Software Updates
Firmware Updates
Vulnerability Analysis
Target-system Identification
Mitigation Recommendations

ABSTRACT

This paper presents a study of update-related vulnerabilities based on a curated dataset of disclosures from the past decade. We analyze the exploitability, impact, and severity of these vulnerabilities, comparing them with the broader population of disclosed vulnerabilities. Our findings reveal that update-related vulnerabilities tend to be more severe and impactful, with a higher concentration of high-severity scores and a greater compromise of confidentiality, integrity, and availability. In contrast to general vulnerabilities, which are often network-exploitable, most update-related attacks require local access. We identify and categorize the most common weakness types in update mechanisms. We found that the most frequent weaknesses include authentication and authorization-related weaknesses, input-related weaknesses, path-related weaknesses, and certificate-related weaknesses. Furthermore, we introduce a rule-based approach with predefined keywords that assign vulnerabilities to six target-system classes. The identification relies on textual vulnerability descriptions together with vendor and product metadata. Finally, based on our analysis, we derive a set of clear mitigation recommendations to help secure update processes and reduce associated risks.

1. Introduction

Software and firmware updates are fundamental to the security and longevity of modern computing systems, from smart home IoT devices to enterprise systems and ubiquitous computing systems. These updates delivered remotely or applied locally are intended to patch vulnerabilities, introduce new features, and ensure reliability across sectors such as healthcare, transportation, and critical infrastructure.

Despite their importance, the update process itself has emerged as a critical attack surface [41]. Misconfigurations, insufficient security controls, and negligence in established best practices frequently expose update mechanisms to exploitation [35]. The consequences can be severe, ranging from unauthorized access and data breaches to system failures. Additionally, several mechanisms of software updates do not have proper security controls [3, 8], do not adhere to best security practices [6], or agree in providing consistency in reporting disclosed vulnerabilities [33]. The global disruption caused by a faulty CrowdStrike update in July 2024, which triggered widespread system crashes across essential services, starkly illustrates the operational and security risks inherent in flawed update processes.

While prior research has proposed methods for securing updates [2, 37, 41], vulnerabilities for software update deployments continue to increase, leading to financial, legal, and reputational damage.

In this paper, we present an empirical analysis framework for studying software and firmware update-related vulnerabilities. Using a curated dataset of update-related vulnerability disclosures from the past decade, we analyze the frequency and severity of these vulnerabilities, their predominant attack vectors, and the most common weakness

classes and adversary capabilities, along with representative exploitation scenarios.

Additionally, we introduce a rule-based approach to classify vulnerabilities into six target-system classes (Home, SCADA, Enterprise, Mobile, PC, Other) using textual vulnerability descriptions with their corresponding vendor names and product metadata. Finally, we derive concrete recommendations for practitioners and maintainers to reduce the security risks inherent in update processes.

Our results show that update-related vulnerabilities tend to be more impactful than the general population of disclosed vulnerabilities. Additionally, the findings indicate that update-related vulnerabilities are primarily compromised locally, whereas overall vulnerabilities are more frequently exploited through network-based attacks. The term "*update process*" will be used hereafter to cover all cases of over-the-air software and firmware updates (including upgrades or downgrades). While the term *overall* will refer to the last decade.

In summary, our contributions are:

- We present an in-depth analysis of software and firmware update vulnerabilities, comparing their exploitability patterns, impact, and severity, to general vulnerabilities observed over the past decade (cf. §4).
- We examine common update-related weaknesses and highlight representative cases, emphasizing how adversaries can leverage these vulnerabilities (cf. §5).
- We introduce a rule-based approach to categorize update-related vulnerabilities into different system classes, using textual descriptions together with vendor and product metadata (cf. §6).
- We propose a set of recommendation strategies designed to mitigate the root causes of update-related weaknesses and enhance system resilience (cf. §7).

*Corresponding author

 ahmad.usman@liu.se (A.B. Usman); mikael.asplund@liu.se (M. Asplund)

ORCID(s): 0000-0001-6781-1420 (A.B. Usman); 0000-0003-1916-3398 (M. Asplund)

- We have made our implementation publicly available, together with a curated dataset and supporting analysis artifacts, to encourage further research and reproducibility in the study of software and firmware update security¹.

The remainder of this paper is organized as follows: Section 2 reviews related studies. Section 3 describes our data collection process and methodology. Section 4 presents an analysis of update-related vulnerabilities. Section 5 provides an analysis of update-related weakness. Section 6 outlines our rule-based approach for vulnerable target-systems. Section 7 introduces the recommendation framework for secure updates. Section 8 discusses the results, limitations and future work. Finally, Section 9 concludes the paper.

2. Related Work

To our knowledge, this is the first work that empirically analyzes software and firmware update vulnerabilities and identifies the most common weaknesses in computing systems. There are several works exploring vulnerabilities of software and firmware updates. Frank Li and Vern Paxson [20] present an empirical study on security patches, analyzing vulnerabilities across various open-source software projects. The study integrates data from the National Vulnerability Database (NVD) to analyze the life cycle of security patches, focusing on the duration of vulnerabilities, patch timeliness, and the reliability of fixes, but does not consider firmware vulnerabilities. Bradley and Barrera [6] examine the IoT software update landscape by analyzing IoT device traffic at the network level. The findings indicate that vendors are not adhering to security best practices. The authors in [3] present challenges with software updates, along with recommendations on how to carry out secure updates. However, their focus is mostly on operating systems, and their recommendations are generic rather than focused on addressing a particular weakness. The authors in [26] study software update practices by analyzing User Agent strings in network traffic, on four software components, namely cURL, Wget, OkHttp, and python-requests. The findings reveal that IoT devices often run outdated and vulnerable versions, emphasizing that vendors frequently deploy older versions that fail to address critical CVEs. However, their analysis focuses on plain-text HTTP traffic, which may not capture the complete software environment on IoT devices.

Setayeshfa et al. [28] highlight the risks associated with software updates, particularly their delays, which leave systems vulnerable. However, they focus on addressing the challenges of investigating update behaviors due to the lack of vendor release schedules and the difficulty of tracking user update installations, with data collection limited to three years. The authors in [32] present the impact of upgrading software vulnerabilities and their severity. Their primary aim is to propose a new method for assessing the vulnerability based on the CVSS.

¹<https://github.com/abusm4n/updatOR>

There has been extensive research dedicated to investigating vulnerabilities in firmware updates.

Recently, Wu et al. [41] introduced a method to detect vulnerabilities in firmware updates. The authors argue that update procedures for firmware introduce new attack surfaces. Through control-flow analysis, they identify and resolve the program execution paths during the update process. In relation to our work, the authors identified the Top 10 firmware update-related weaknesses, some of which overlap with our findings; however, their study is limited in scope to firmware vulnerabilities.

Ibrahim et al. [16] conduct a systematic security analysis of firmware update mechanisms in IoT devices facilitated by a companion application. They define a threat model, categorize potential security issues, and then analyze 23 popular IoT devices to identify vulnerabilities. Their results indicate that most of these devices are vulnerable due to the adoption of insecure firmware update mechanisms. Costin et al. [9] provide an analysis of firmware images. Through their analysis, new insights into the security of embedded devices were revealed. Bideh and Gehrman [23] revisit the firmware upgrade challenge and present RoSym as a proposed solution. However, their study specifically focuses on building a symmetric key with the limitation of protecting only against DoS attacks. Finally, Froh et al. [15] propose a mitigation method for detecting malicious updates. However, it uses solely differential static analysis exclusively on open-source packages.

3. Methodology

This section outlines the methodology employed in this study, including the formulation of the research questions, the sources of vulnerability data, the criteria used for selecting update-related vulnerabilities, the analysis of relevant security metrics, and the mapping of CVEs to CWEs to synthesize findings and evaluate the stated hypotheses.

3.1. Research Questions

To guide this study and structure our analysis, we investigate the following **Research Questions (RQs)**:

- **RQ1:** How do update-related vulnerabilities differ from other general vulnerabilities in terms of their exploitability characteristics, impact on confidentiality, integrity, and availability, and their severity level? To answer this question, we analyze vulnerabilities affecting update systems and compare them with the overall set of vulnerabilities disclosed over the past decade, identifying key similarities and differences across exploitability (such as attack vector and attack complexity), and impact with regards to confidentiality, integrity, and availability, and severity level distributions.
- **RQ2:** What are the most common weaknesses underlying update-related vulnerabilities, and how do they compare to those observed in vulnerabilities overall?

We map update-related CVEs to their corresponding CWE categories to identify the most prevalent weaknesses and discuss their relevance, and show how attackers exploit them within software and firmware update processes.

- **RQ3:** Which target-system categories are most frequently affected by update-related vulnerabilities, and how does this distribution differ from the overall vulnerability dataset? We classify vulnerabilities into six target-system categories using vulnerability description, vendor, and product metadata, then compare class distributions and model performance between update-related and overall datasets.
- **RQ4:** What mitigation measures should be prioritized to reduce the risk of update-related vulnerabilities based on the identified weaknesses? Drawing on our findings, we propose a recommendation matrix for secure update mechanisms, outlining practical mitigation strategies aimed at addressing the most critical and recurring update-related weaknesses.

These research questions are important because update mechanisms are widely deployed and can have severe consequences when compromised. Investigating how update-related vulnerabilities differ from the broader vulnerability landscape (RQ1), identifying the most prevalent weaknesses (RQ2), and determining which target-system classes are most exposed (RQ3) provides empirical evidence for risk prioritization and helps guide the selection of appropriate mitigation measures (RQ4).

The overall study methodology is illustrated in Figure 1, which provides an overview of four main phases.

In Phase 1, update-related vulnerabilities are identified from the official CVE repository using a keyword-based filtering strategy, followed by manual validation to remove false positives and produce a curated dataset of software and firmware update vulnerabilities (cf. §3.2). Having data collection and curation completed, this leads to two phases. Phase 2 performs the core vulnerability analysis, where CVSSv2/v3 metrics are extracted and compared against the overall population, and vulnerabilities are mapped to CWEs to identify dominant weakness patterns, which are subsequently grouped into five high-level weakness categories. Phase 3 introduces a system that assigns each vulnerability to a target system class using a rule-based approach, resulting in a labeled dataset for downstream analysis. Followed by evaluation and comparative analysis. Finally, in Phase 4, the findings are synthesized by deriving mitigation recommendations from the identified weaknesses and constructing a recommendation matrix that directly addresses the observed update-related risks. These four phases are further explained in the following subsections.

3.2. Data Collection and Curation (Phase 1)

We primarily collected data from the CVE List official repository [10, 11]. We downloaded vulnerability incidents

disclosed and selected those from the past decade, as shown in Figure 2. We restrict our analysis to the period between 2016 and 2025, as recent vulnerabilities provide more precise scoring metrics and detailed information than older ones. We can see that there is an increasing number of vulnerabilities disclosed every year.

To be consistent with the data in our experiments, the statistics in the figure are based on the vulnerability files found in the CVE List official repository, and we used the repository so that we can be able to pull new updates (Commit ID: 174e04d) done by the CVE developers.

3.2.1. Extracting Update-related Vulnerabilities

The extraction of update-related vulnerabilities was conducted using a semi-automated curation process combining keyword-based retrieval with manual validation. We first collected vulnerability records from 2016 to 2025, comprising a total of 243,610 vulnerabilities. The annual count of these overall vulnerabilities is shown in Figure 2, where the data are plotted on a logarithmic scale to facilitate the comparison of growth rates across different magnitudes.

To identify candidates potentially related to software update vulnerabilities, we applied an automated keyword search over vulnerability descriptions. The query included terms associated with software and firmware updates and their execution phases. These keywords are: software update, firmware update, software upgrade, firmware upgrade, update process, upgrade process, downgrade, and OTA update or over-the-air update. The overall retrieval workflow is summarized in Phase 1 Figure 1.

3.2.2. Manual Validation

Because these keywords may also appear in vulnerabilities unrelated to the update process, we performed a manual review of all retrieved entries to eliminate false positives. Each candidate vulnerability was evaluated against a set of predefined selection criteria to determine whether it genuinely affected the update mechanism. This step ensured that vulnerabilities sharing similar terminology but unrelated to update behavior were excluded from the final dataset.

We categorized confirmed update-related vulnerabilities into two primary classes: software update-related and firmware update-related. When a vulnerability applied to both categories, it was assigned to the most appropriate class and removed from the other to avoid duplication and bias in subsequent analyses.

For this study, a vulnerability is classified as update-related if it involves defects or malicious activity affecting the update process in one or more of the following ways: (i) compromising the integrity of the update itself, including tampered or malicious updates; (ii) exploitation during update application, such as man-in-the-middle attacks during update upload or download; (iii) vulnerabilities arising from upgrade or downgrade procedures; or (iv) flaws in the logic, permissions, or management of the update mechanism, including weak access controls or insecure update execution.

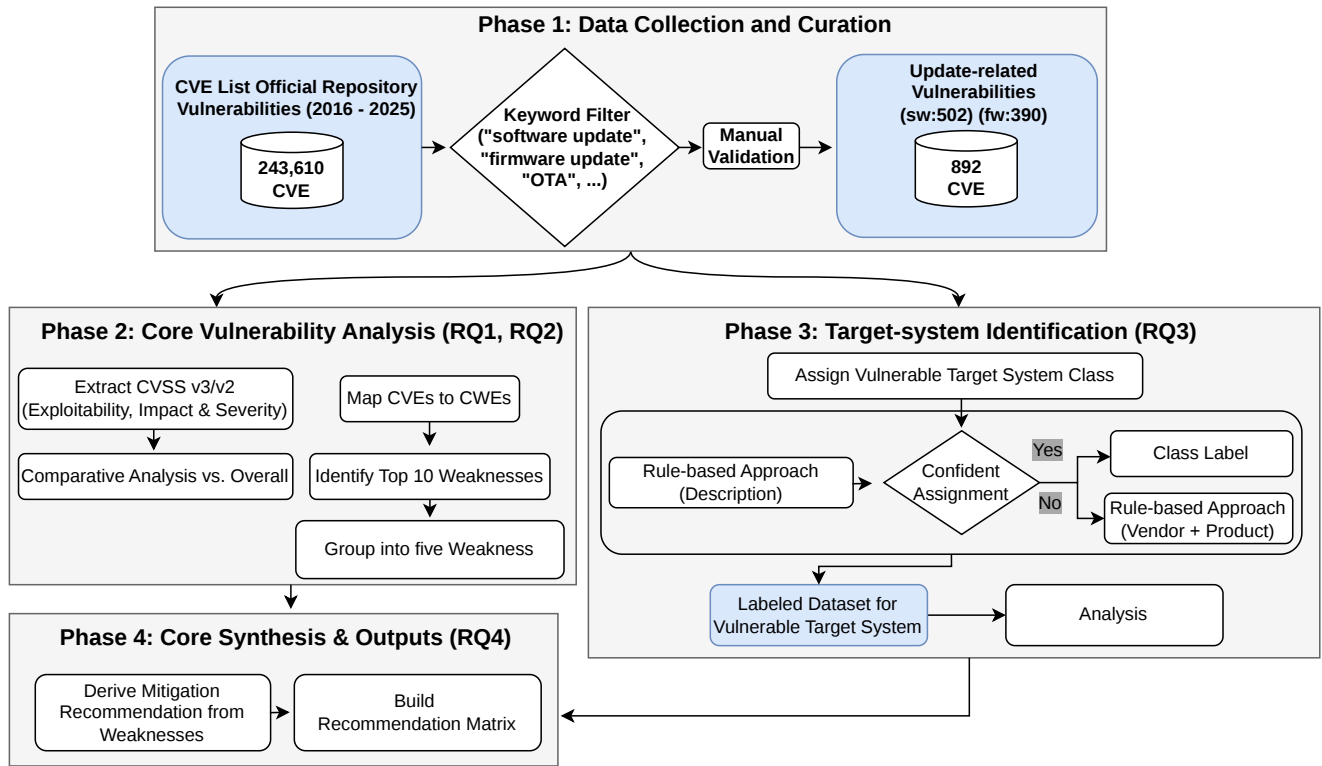


Figure 1: Overview of the methodology, showing the four study phases: (i) data collection and curation, (ii) core vulnerability analysis, (iii) target-system identification, and (iv) synthesis of mitigation recommendations.

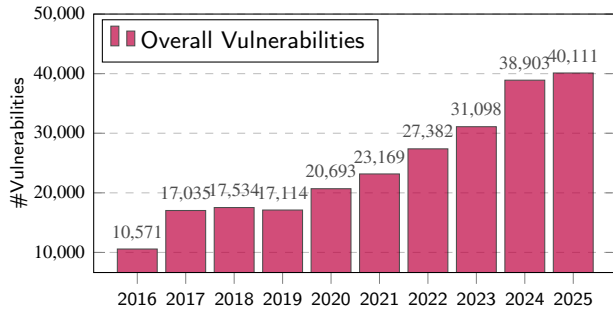


Figure 2: Overall CVE count per year.

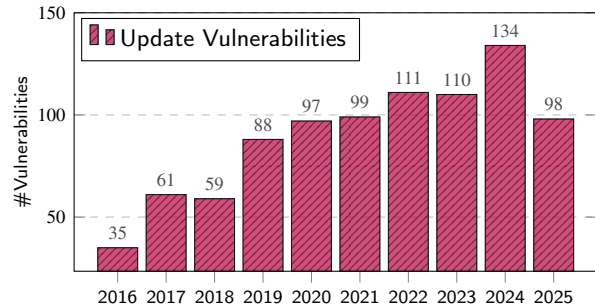


Figure 3: Update-related CVE count per year.

Conversely, vulnerabilities are excluded if exploitation occurs outside the update process itself. For example, if an attacker compromises a system after an update is complete, it falls outside our paper's definition of an update-related vulnerability. Similarly, we exclude cases where an update was provided to fix a security issue, or where an attacker exploits weaknesses in an application's standard operation or authentication mechanisms.

After manual validation and the removal of false positives, we obtained a curated dataset of 892 vulnerabilities, of which 502 are related to software updates and 390 are related to firmware updates. This dataset is used for the remainder of the analysis in this paper.

We separate software vulnerabilities from firmware vulnerabilities due to their distinct nature during the update

process. The former typically occur at the application layer, while the latter are embedded at lower-level system components, such as microcontroller firmware.

Figure 3 presents the temporal distribution of update-related vulnerabilities we found in our analysis. The figure shows a steady increase in reported update-related vulnerabilities over recent years, except in 2025. This deviation is likely because many vulnerabilities from the current year are still under review or have not yet been reported. Overall, this trend highlights the growing security relevance of update mechanisms and underscores the need for focused analysis in this domain.

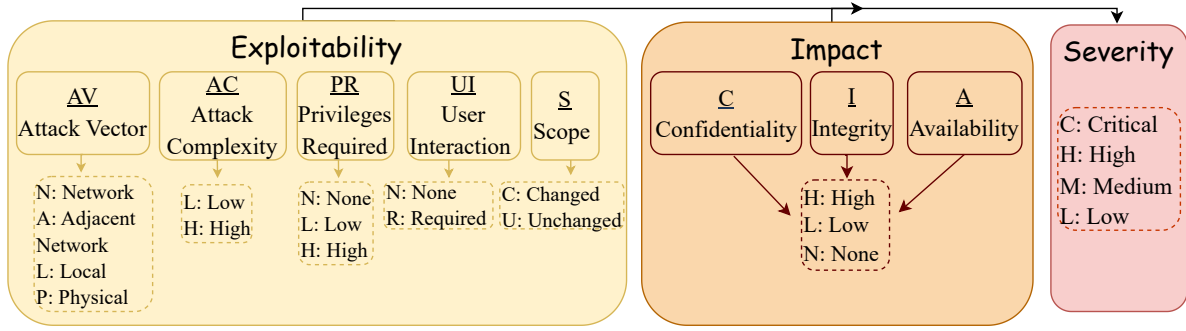


Figure 4: CVSS v3 Metrics and Sub-metrics.

3.3. Core Vulnerability Analysis (Phase 2)

To investigate potential differences between update-related vulnerabilities and the overall vulnerabilities in terms of severity, we rely on the CVSS v3 base score metrics, which is maintained and standardized by the Forum of Incident Response and Security Teams (FIRST) [14]. This methodology includes three main components: exploitability metrics, impact metrics, and severity calculation [14], as illustrated in Figure 1.

3.3.1. Selected Metrics

Exploitability Metrics: quantifies how easily a vulnerability can be exploited. The score is computed as: $\text{Exploitability} = 8.22 \times AV \times AC \times PR \times UI$. And its component consists of five sub-metrics defined as follows:

- **Attack Vector (AV):** Describes the means by which a vulnerability can be exploited, classified as Network (N), Adjacent Network (A), Local (L), or Physical (P).
- **Attack Complexity (AC):** Represents the difficulty of exploiting the vulnerability, categorized as Low (L) or High (H).
- **Privileges Required (PR):** Indicates the level of privileges an attacker must possess before exploitation, defined as None (N), Low (L), or High (H).
- **User Interaction (UI):** Specifies whether exploitation requires user involvement, either Required (R) or None (N).
- **Scope (S):** Describes whether exploitation can impact components beyond the initially vulnerable one, classified as Unchanged (U) or Changed (C).

Impact Metrics: capture the consequences of a successful exploitation and consist of three sub-metrics:

- **Confidentiality (C):** Evaluates the extent to which sensitive information may be disclosed.
- **Integrity (I):** Measures the degree to which resources can be modified, corrupted, or tampered with

- **Availability (A):** Assesses the impact on system availability, such as service disruption or denial of service.

Severity Metrics (Base Score): quantifies the intrinsic severity of a vulnerability based on its impact and exploitability, depending on the Scope of the vulnerability

The exploitability and impact characteristics are combined by CVSS to compute a numerical base score ranging from 0 to 10, representing the overall **severity** of the vulnerability. The resulting score is then mapped to qualitative severity levels: Low (0.1-3.9), Medium (4.0-6.9), High (7.0-8.9), and Critical (9.0-10.0).

To illustrate this, consider the update-related vulnerability CVE-2025-56513, which is assigned the vector string `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`. This indicates that the vulnerability is remotely exploitable over the network, has low attack complexity, requires no privileges or user interaction, and has a high impact on confidentiality, integrity, and availability.

In cases where CVSS v3 scores are unavailable, we rely on the corresponding CVSS v2 scores. Although both versions assess vulnerability severity using exploitability and impact dimensions, CVSS v3 replaces the Authentication metric with Privileges Required, explicitly models User Interaction and Scope, and extends the Attack Vector to include physical access. In addition, CVSS v2 represents impact using coarse None/Partial/Complete levels, whereas CVSS v3 adopts more granular None/Low/High assessments for confidentiality, integrity, and availability. Despite these structural differences, the use of CVSS v2 scores does not materially affect the aggregate trends or conclusions of our analysis.

3.3.2. Mapping CVEs to CWEs

To quantify the prevalence of different vulnerability types in our dataset, we deploy a formal mapping between CVEs and CWEs. Let $\mathcal{J}$ represent the set of all CVEs in our analysis, and let $\mathcal{I}$ represent the set of all CWE identifiers present in the dataset. For each CWE $i \in \mathcal{I}$, we compute N_i , the total number of CVEs associated with that specific weakness type.

This relationship is captured using a binary indicator variable e_{ij} , defined as:

$$N_i = \sum_{j \in J} e_{ij}, \quad e_{ij} = \begin{cases} 1, & \text{if CVE } j \text{ maps to CWE } i \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

Note that some vulnerabilities in the dataset are associated with more than one value of exploitability, impact, and base score; this is because some CVEs have multiple versions affected. For this reason, we opted to consider the most critically affected versions.

3.4. Target-system Identification Framework (Phase 3)

To identify CVEs according to their target systems, we adopt the classification scheme proposed in [5], which divides systems into six classes based on functionality and deployment environment:

- **Home:** Consumer devices, including smart TVs, home routers, smart appliances, and household IoT devices.
- **SCADA:** Industrial control and operational technology (OT) systems, including ICS/SCADA components, automation controllers, and RTUs.
- **Enterprise:** Enterprise and service-provider infrastructure, including data-center systems, network and security appliances, and cloud platform components.
- **Mobile:** Mobile and wearable platforms, including smartphones, tablets, and smartwatches.
- **PC:** End-user general-purpose computing systems, including desktops, laptops, and workstation hosts.
- **Other:** Devices and systems that do not clearly fit the above classes, including mixed-purpose peripherals and niche or specialized platforms.

This categorization provides a structured basis for mapping vulnerabilities to target systems and enables more precise vulnerability analysis and risk assessment across heterogeneous environments.

3.4.1. Rule-based Approach

We implement a rule-based classifier as an interpretable baseline. For each of the six target-system classes, we define a set of regular expression patterns derived from the characteristic vocabulary associated with that class.

The patterns are constructed by inspecting a random sample of CVE descriptions and are organized around the following semantic groups. **Home:** Patterns target consumer-grade connected devices, including smart-home appliances (*smart thermostat, smart lock, home automation*), networking equipment (*router, wireless access point*), IP cameras and baby monitors, home entertainment systems (*smart TV, streaming device, gaming console*), and well-known consumer vendors (e.g. TP-Link, D-Link). **SCADA:** Patterns capture industrial and operational technology terminology, including control-system acronyms (*SCADA, ICS, PLC,*

HMI, RTU). **Enterprise:** Patterns identify enterprise-grade infrastructure, including network and security appliances (*firewall, VPN, switch, load balancer*), server and cloud platforms (*data center, cloud infrastructure*), and major enterprise vendors (e.g. Cisco, Juniper, Fortinet, VMware, Oracle). **Mobile:** Patterns match mobile and wearable platforms, including identifiers such as *Android, iOS, iPhone*, and *iPad*, distribution channels (*Google Play, App Store*), and wearable devices (*smartwatch, fitness tracker*). **PC:** Patterns capture personal and general-purpose computing systems, including device form factors (*desktop, laptop, personal computer*), desktop operating systems (*Windows, Linux, macOS, Ubuntu, Debian, Red Hat*), hardware components (*CPU, GPU, SSD*), and common PC software (*Microsoft Office, antivirus*).

The classifier assigns scores to each class based on case-insensitive regex matches in the vulnerability description. To prioritize stronger signals, matches are weighted as follows: multi-word patterns receive a weight of 3, while single-token patterns receive a weight of 2.

To reduce the number of vulnerabilities assigned to the *Other* class and to capture manufacturer-specific signals, we extend the approach by incorporating vendor and product metadata extracted from the affected field of the CVE record. A vendor-name match against a curated per-class list contributes +3, a product-keyword match in the product field contributes +2, and a product-keyword match in the description contributes +1.

In addition, the label produced by the description, only classifier is used as a soft vote, contributing +2 to the corresponding class score. The class with the highest accumulated score is selected when it exceeds a predefined decision threshold (score ≥ 2); otherwise, the vulnerability is labeled *Other*. The complete rule-based approach is available in the artifact accompanying this paper.

4. Update-related Vulnerabilities (RQ1)

This section synthesizes the update-related vulnerabilities in order to answer **RQ1**, to determine whether there are significant differences between update-related vulnerabilities compared to other types of vulnerabilities. We analyzed update-related vulnerabilities to characterize exploitation methods in terms of attack vectors, attack complexity, required privileges, user interaction, and scope, as well as their impact on confidentiality, integrity, and availability, and overall severity. We then compare these findings with those of all reported vulnerabilities over the past decade.

For a structured presentation of our results, we adopt the organization illustrated in Figure 4 and grounded in the CVSS v3 framework (cf. §3.3). Specifically, we first analyze exploitability, then impact, and finally severity.

4.1. Exploitability

To assess exploitability, we first present the distributions of attack vectors and attack complexities for software update vulnerabilities, firmware update vulnerabilities, and the overall vulnerability set, as shown in 5, 6, and 7.

Understanding Security Weaknesses in Software and Firmware Updates

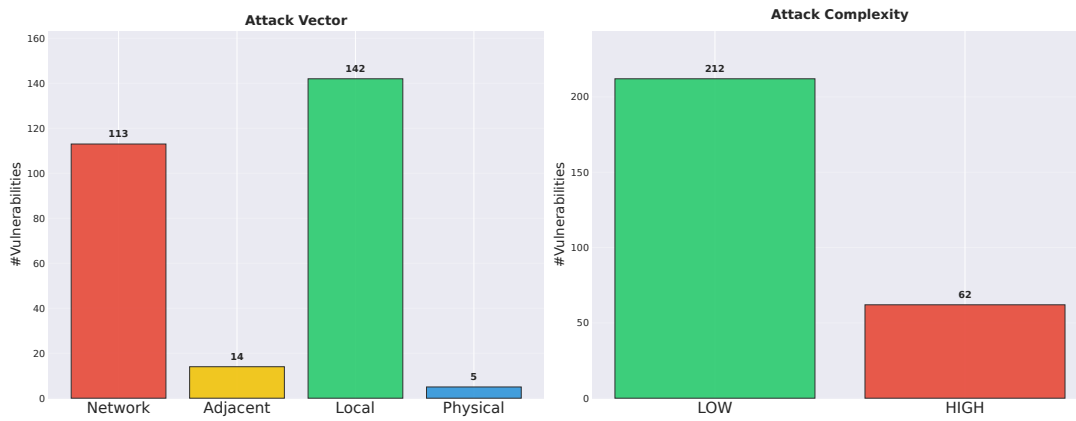


Figure 5: Software Updates Attack Metrics.

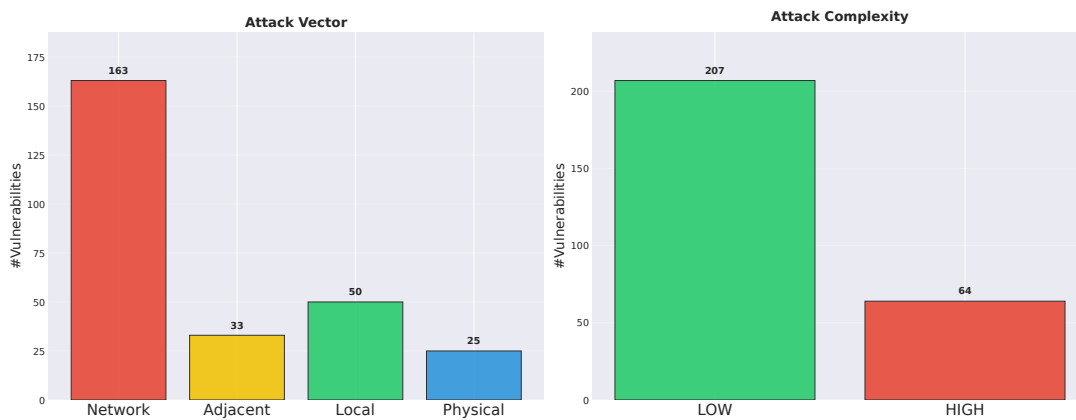


Figure 6: Firmware Updates Attack Metrics.

Our analysis shows that software update vulnerabilities are predominantly exploited through local attack vectors, followed by network, adjacent-network, and physical vectors. Conversely, in the overall vulnerabilities, network-based exploitation is the most prevalent, with local, adjacent-network, and physical attacks occurring less frequently. Notably, the distributions of attack complexity exhibit similar

patterns across update-related vulnerabilities and the overall vulnerability set.

To further assess exploitability, we present the remaining three sub-metrics (Privileges Required, User Interaction, and Scope), illustrating their distributions for update-related and the overall vulnerability set in Figures 8 and 9. For update-related vulnerabilities (Figure 8), most can be exploited with

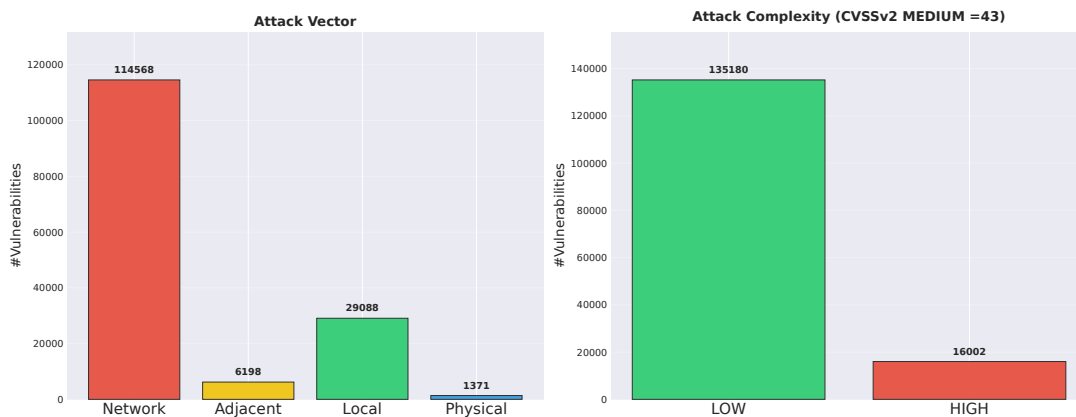


Figure 7: Overall Attack Metrics.

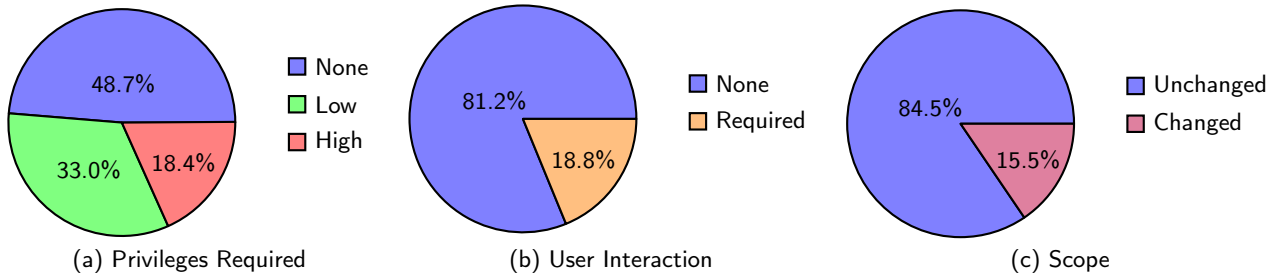


Figure 8: Update-related Vulnerability Distribution by Privileges Required, User Interaction, and Scope.

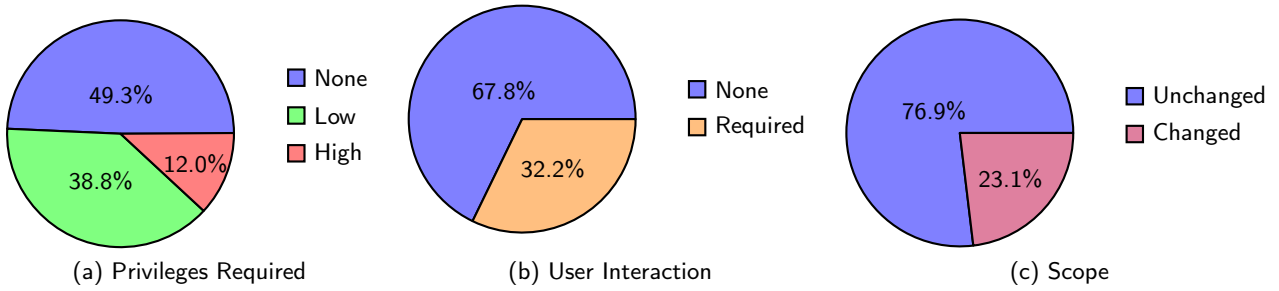


Figure 9: Overall Vulnerability Distribution by Privileges Required, User Interaction, and Scope.

minimal or no privileges and often do not require any user interaction, making them particularly easy to target. While scope changes are less common, their presence indicates that some update vulnerabilities could affect components beyond the directly targeted system. In contrast, the overall vulnerability set (Figure 9) exhibits a more varied distribution. User interaction and scope changes occur more frequently than in update-related vulnerabilities, although the differences are proportional rather than absolute. Privilege requirements are generally similar between the two sets. Therefore, update-related vulnerabilities appear easier to exploit in practice than the overall vulnerability population.

4.2. Impact

To evaluate the impact of update-related vulnerabilities, we present violations of confidentiality, integrity, and availability. Figures 10 and 11 present a comparative analysis of the impact of vulnerabilities on the confidentiality, integrity, and availability triad. Figure 10 focuses exclusively on update-related vulnerabilities, while Figure 11 summarizes the overall vulnerabilities.

For update-related vulnerabilities, the results show a pronounced concentration of high-impact violations across all three dimensions. Specifically, confidentiality is most severely affected, with the majority of update-related vulnerabilities exhibiting a high confidentiality impact, substantially outweighing those with low or no impact. A similar trend is observed for integrity, where high-impact violations dominate. Availability is likewise heavily impacted, with vulnerabilities classified as having high availability impact, reflecting the potential for denial-of-service or persistent system disruption through malicious updates.

In contrast, the overall vulnerability (11) shows a more balanced distribution across the high, low, and no-impact

categories for confidentiality, integrity, and availability. Although the absolute number of high-impact vulnerabilities is larger due to the size of the dataset, high-impact cases do not dominate uniformly across all dimensions. For example, while confidentiality shows a substantial number of high-impact vulnerabilities, integrity and availability impacts are more evenly spread between high, low, and none categories, suggesting a less consistently severe security posture across general vulnerabilities.

Table 1 compares the distribution of impact levels for confidentiality, integrity, and availability between update-related vulnerabilities and the overall vulnerability. The color intensity highlights increasing severity from none to critical, enabling a direct visual comparison of impact concentration across categories.

Across all three impact dimensions, update-related vulnerabilities are dominated by high-severity impacts. For confidentiality, 66.4% of update-related vulnerabilities are classified as high impact, with an additional 2.0% reaching critical severity. Integrity exhibits a nearly identical pattern, with 65.5% high-impact and 2.0% critical cases. Availability is likewise severely affected, with 61.8% of update-related vulnerabilities rated as high impact and 2.0% as critical, reflecting the potential for widespread service disruption or permanent system disablement through malicious or faulty updates.

In contrast, the overall vulnerability shows a lower distributed impact profile. High-impact cases account for only 39.4% (confidentiality), 33.0% (integrity), and 36.6% (availability), while a larger fraction of vulnerabilities fall into the low or no-impact categories. Notably, the proportion of vulnerabilities with no impact is consistently higher in the overall dataset, reaching 37.1% for availability, compared to 28.1% for update-related vulnerabilities.



Figure 10: Update-related Impact on Confidentiality, Integrity and availability.

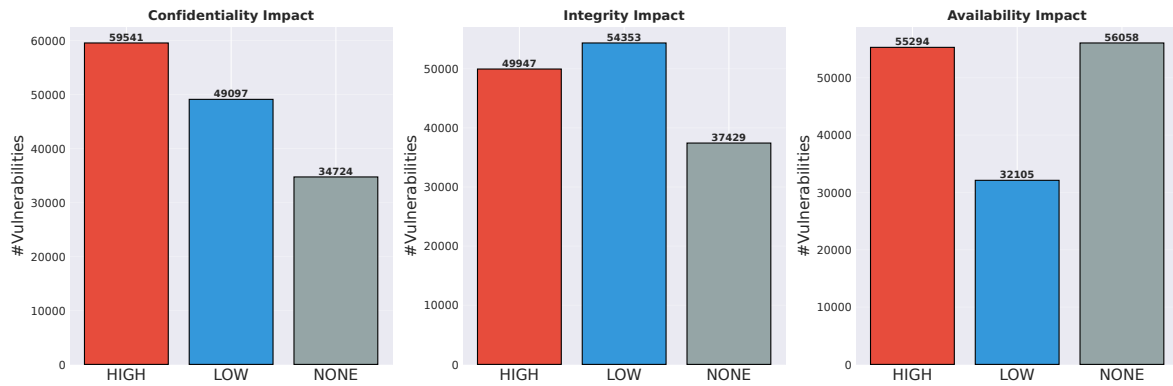


Figure 11: Overall Impact on Confidentiality, Integrity, and Availability.

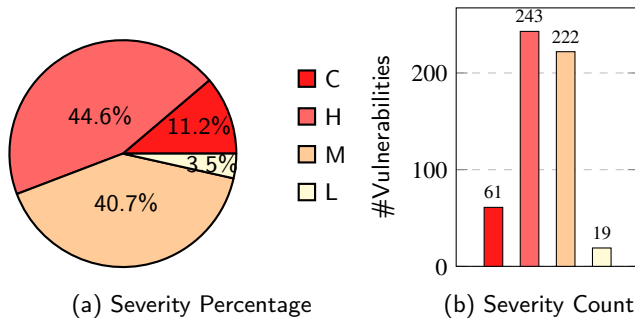


Figure 12: Update-related Severity Distribution and Vulnerability Absolute Counts:(C:Critical, H:High, M:Medium, L:Low).

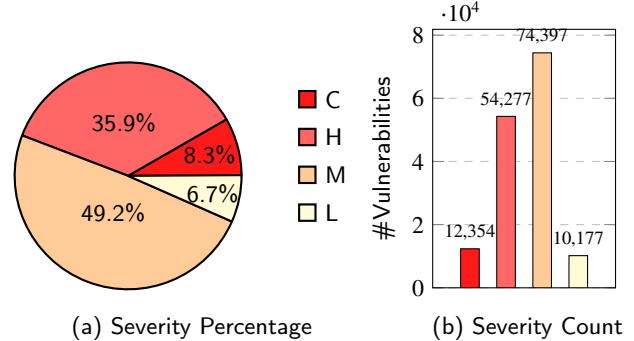


Figure 13: Overall Severity Distribution and Vulnerability Absolute Counts:(C:Critical, H:High, M:Medium, L:Low).

This comparison demonstrates that update-related vulnerabilities are more (impactful) severe than vulnerabilities in general.

4.3. Severity

Figures 12 and 13 compare the severity distributions of update-related vulnerabilities against the overall vulnerability, using both relative percentages and absolute counts. While the overall dataset is substantially larger in volume, the update-related subset exhibits a markedly more severe and tightly clustered impact profile. As shown in Figure

12, update-related vulnerabilities has higher severity vulnerabilities constitute the largest share (44.6%), followed by critical vulnerabilities (11.2%), resulting in more than half of update-related vulnerabilities (55.8%) being classified as high or critical. Medium-severity cases account for 40.7%, while low-severity vulnerabilities are rare (3.5%), indicating a limited presence of benign or low-risk update flaws.

In contrast, the overall vulnerability distribution in Figure 13 with medium severity is centered at 49.2%, with high-severity vulnerabilities representing a smaller fraction

Table 1Update-related vs. Overall Vulnerability Impact Metrics Comparison. **Bold #** Indicates update-related more critical and higher.

Impact	Update-related %					Overall %				
	Critical ^{v2}	High	Partial ^{v2}	Low	None	Critical ^{v2}	High	Partial ^{v2}	Low	None
Confidentiality	2.0	66.4	0.9	10.5	20.2	0.7	39.4	4.5	32.5	23.0
Integrity	2.0	65.5	0.9	14.7	16.9	0.7	33.0	5.6	35.9	24.8
Availability	2.0	61.8	0.7	7.3	28.1	0.8	36.6	4.4	21.2	37.1

Note: Critical^{v2} and Partial^{v2} impact belongs to CVSSv2 metrics, detailed for %completeness.

(35.9%) and critical cases accounting for only 8.3%. Low-severity vulnerabilities are also more prevalent in the overall dataset (6.7%), reflecting a broader and less concentrated severity profile. Taken together, only 44.2% of all vulnerabilities fall into the high or critical categories, compared to nearly 56% for update-related vulnerabilities.

The absolute counts further reinforce this contrast. Despite the comparatively small number of update-related vulnerabilities, their severity is disproportionately high, with critical and high cases dominating the distribution relative to low-severity ones.

Overall, these findings demonstrate that update-related vulnerabilities are more critical and highly violated than vulnerabilities in general.

5. Update-related Weaknesses (RQ2)

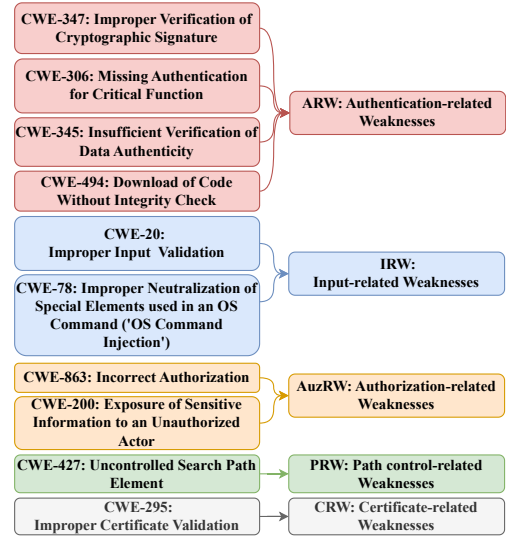
This section identifies the weaknesses most frequent in update-related vulnerabilities and compares them with those observed overall to address **RQ2** (What are the most common weaknesses of update-related vulnerabilities?). It further explains how these weaknesses can be exploited and outlines realistic scenarios in which such attacks may occur. Several of the identified weaknesses were classified by MITRE as Top 25, Stubborn, or Most Dangerous weaknesses in 2023 [13].

By analyzing the metric fields used to map CVEs to CWEs (cf. §3.3), we extracted the most frequently reported weaknesses associated with the update process. Table 2 summarizes the ten most commonly exploited update-related weaknesses and compares them with the most prevalent weaknesses overall.

To structure the analysis, we group related weaknesses into five categories based on the highest severity: Authentication-related Weaknesses (ARW), Authorization-related Weaknesses (AuzRW), Input-related Weaknesses (IRW), Path control-related Weaknesses (PRW), and Certificate-related Weaknesses (CRW). Each group is highlighted using a distinct color in the table, as illustrated in Figure 14.

The results show that ARW dominates update-related vulnerabilities. IRW is the second most common, followed by AuzRW, PRW, and CRW.

In contrast, Cross-Site Scripting (XSS) and SQL injection are more strongly associated with vulnerabilities in the overall dataset rather than in update-specific cases.

**Figure 14:** Grouping Weaknesses into Five Closely-related.

The Top 10 proportion $\oplus$ is 37.38% for update-related vulnerabilities, compared to 44.35% for the overall dataset, as shown in Table 2.

The following sections provide further context on how these weaknesses arise during the update process and present real-world examples from our curated dataset to demonstrate the impact and attacker capabilities enabled by these vulnerabilities.

5.1. Authentication-related Weaknesses (ARW)

Authentication-related weaknesses are the most prominent category in update-related vulnerabilities. This group includes four recurring weaknesses that undermine the authenticity and integrity of software updates. Improper Verification of Cryptographic Signature (CWE-347), Missing Authentication for Critical Function (CWE-306), Insufficient Verification of Data Authenticity (CWE-345), and Download of Code Without Integrity Check (CWE-494):

These weaknesses occur when a product fails to properly verify the authenticity or origin of update-related data. This includes missing or incorrect verification of cryptographic signatures, as well as the execution of update code obtained from untrusted sources.

Table 2

Top 10 Weaknesses of Update-related vs. Overall Vulnerabilities.

#	Update-Related				Overall			
	CWE-ID	Related Issue	%	⊕%	CWE-ID	Related Issue	%	⊕%
1	CWE-347	Cryptographic Signature	6.73	6.73	CWE-79	Cross-site Scripting	15.5	15.5
2	CWE-306	Authentication	4.67	11.40	CWE-89	SQL Injection	6.79	22.29
3	CWE-345	Data Authenticity	4.11	15.51	CWE-352	Cross-Site Request Forgery	3.48	25.77
4	CWE-78	OS Command Injection	4.11	19.62	CWE-862	Missing Authorization	3.30	29.07
5	CWE-494	Integrity Check	3.74	23.36	CWE-787	Out-of-bounds Write	2.74	31.81
6	CWE-20	Input Validation	3.18	26.54	CWE-20	Input Validation	2.74	34.55
7	CWE-427	Path Element	2.80	29.34	CWE-125	Out-of-bounds Read	2.61	37.16
8	CWE-863	Incorrect Authorization	2.80	32.14	CWE-22	Path Traversal	2.50	39.66
9	CWE-200	Unauthorized Actor	2.62	34.76	CWE-284	Access Control	2.39	42.05
10	CWE-295	Certificate Validation	2.62	37.38	CWE-74	Injection	2.30	44.35

In the context of software updates, such weaknesses typically arise when a device either does not verify or incorrectly verifies the authenticity of the update binary. This may occur during the download phase or before deploying an update that already resides on the device. For example, in CVE-2023-4589 → (CWE-345), the digital signature is absent, and update integrity is not verified during the update process. Similarly, in CVE-2016-6567 → (CWE-494), the download manager fails to validate the authenticity of the firmware before execution, enabling an attacker to inject a malicious update, replace the operating system, or gain root privilege and execute arbitrary code.

5.2. Input-related Weaknesses (IRW)

Input-related weaknesses are the second major class observed in update workflows. They include two groups. Improper Input Validation (CWE-20), Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') (CWE-78): These weaknesses occur when update parameters, metadata, or command inputs are not sufficiently validated before processing.

In update mechanisms, this typically involves insufficient validation of raw input data, such as parameters, strings, or firmware metadata, during the upload or download of updates. For instance, in CVE-2017-6759 → (CWE-20), the update mechanism (UpgradeManager) is vulnerable due to inadequate input validation, allowing manipulation of the upgrade package. Similarly, CVE-2025-52379 → (CWE-78) affects the firmware update feature via the upgradeFileName parameter, leading to OS command injection and enabling attackers to write arbitrary files or execute remote code.

5.3. Authorization-related Weaknesses (AuzRW)

Authorization-related weaknesses represent the third most prominent category in update-related vulnerabilities. This group consists of Incorrect Authorization (CWE-863) and Exposure of Sensitive Information to an Unauthorized Actor (CWE-200): These weaknesses arise when authorization checks are missing, incorrectly implemented, or insufficiently enforced, resulting in unauthorized access to protected resources or sensitive information.

In the context of software updates, such weaknesses occur when update mechanisms fail to properly restrict access to update-related functionality or data, allowing unauthorized actors to interact with the update process or retrieve sensitive information. For example, in CVE-2024-48792 → (CWE-863) and CVE-2024-48799 → (CWE-200), flaws in the firmware update process enable remote adversaries to access sensitive information without proper authorization. Similarly, in CVE-2024-39352 (CWE-863), an attacker can bypass authorization checks for the firmware upgrade feature with root privileges.

5.4. Path control-related Weaknesses (PRW)

Path control-related weaknesses have only one category for update-related weaknesses. Uncontrolled Search Path Element (CWE-427): These weaknesses arise when a product constructs file paths using externally influenced input without adequately sanitizing special path elements, or when it relies on fixed search paths that can partially be controlled by an adversary.

In the context of software updates, such weaknesses occur when a device does not ensure that update files are downloaded to or accessed from the intended directory hierarchy. This can be caused by uncontrolled search paths or insufficient filtering of path elements. Update mechanisms often retrieve update data through multiple access paths, one or more of which may be attacker-controlled. For example, in CVE-2023-32660 → (CWE-427), the firmware update utility uses an uncontrolled search path, enabling an attacker to manipulate file locations and escalate privileges.

5.5. Certificate-related Weaknesses (CRW)

Certificate-related Weaknesses has also only one category for update-related weaknesses, Improper Certificate Validation (CWE-295): These weaknesses occur when a product fails to correctly validate digital certificates, thereby assigning trust to outdated certificates.

In the context of software updates, this typically occurs when certificates used to authenticate update servers or validate update packages are not properly verified. For instance,

in CVE-2022-3913 -> (CWE-295), the update download process fails to validate server certificates, allowing an attacker to perform a man-in-the-middle attack and escalate privileges. While certificate-related weaknesses share similarities with authentication-related weaknesses, we treat them as a distinct category due to the specialized management of certificates and their role in mitigation strategies, particularly those involving remote attestation.

6. Target-system Identification (RQ3)

This section presents the framework implemented in this paper for classifying vulnerabilities based on the affected target-system categories within update-related vulnerabilities and overall for answering RQ3.

6.1. Rule-based Approach

As a first step, we developed a rule-based approach designed to categorize vulnerable target-systems into six predefined classes.

Table 3 provides an excerpt of representative systems associated with software and firmware update vulnerabilities.

Initially, the rule-based approach relies solely on the textual descriptions extracted from disclosed vulnerabilities. However, this approach often results in a significant number of vulnerabilities being classified as Other, due to the absence of distinctive keywords in their descriptions. To address this limitation, we extended the framework to also consider vendor names and product metadata, which improved the accuracy and contextual understanding of the classification.

Figure 16 illustrates the rule-based identification results. The left side of the figure shows the results obtained using only textual descriptions, while the right side presents the improved results after incorporating vendor and product metadata. This combined approach, leveraging descriptions together with vendor and product information, enhances the rule-based method by correctly labeling classes and reducing bias toward the Other category.

Figure 15 presents a heatmap illustrating the overlap of the top 10 CWE categories across five target-system classes. The results reveal notable differences in weakness distributions among system types. For instance, CWE-347 (Improper Verification of Cryptographic Signature) shows the highest overlap with 27 occurrences, and is particularly prominent in PC systems (13 instances), followed by Enterprise environments (7), Home systems (4), and SCADA systems (3).

The next most common weakness is CWE-345 (Insufficient Verification of Data Authenticity) with 16 occurrences, followed by CWE-306 (Missing Authentication for Critical Function) with 13 overlaps.

These findings indicate that most vulnerable target systems are associated with authentication-related weaknesses (ARW), with the exception of the Mobile class.

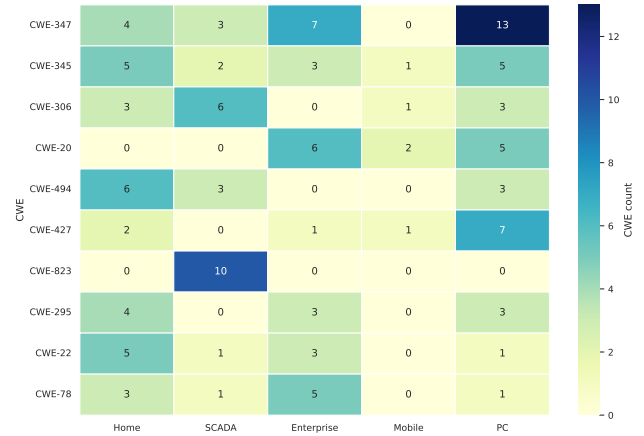


Figure 15: Heatmap of Top 10 Overlap CWE across Target-system Classes.

Table 3
Example of Classification.

Class	Description	Vendor	Product
Home	An exploitable firmware downgrade vulnerability exists in the firmware update functionality of Yi Home Camera ... CVE-2018-3891	Unknown	YiTechnology
SCADA	A vulnerability exists in Schneider Electric's Modicon Quantum... to upgrade the firmware ... CVE-2018-7240	Schneider	Modicon Quantum
Enterprise	IBM Security Secret Server 10.7 processes patches, image backups and other updates ... CVE-2019-4640	IBM	Security Secret Server
Mobile	A elevation of privilege vulnerability in Android system OTA updates... CVE-2017-13265	Google Inc.	Android
PC	An elevation of privilege vulnerability exists in the Microsoft Windows Update... CVE-2020-1014	Microsoft	Windows
Other	ALTOOLS update service 18.1 and earlier versions contains a local privilege escalation... CVE-2019-12808	ESTSOFT	ALTOOLS Update Service

7. Recommendations (RQ4)

This section presents a set of recommendations aimed at mitigating update-related weaknesses. We further map these recommendations to the identified weaknesses, thereby addressing RQ4.

7.1. Mitigation Recommendations

Taken together, the Top 10 weaknesses account for 37.38% (cf. Table 2) of all reported update-related vulnerabilities with associated CWE information. Despite the likely under-reporting of vulnerabilities and the absence of CWE classifications for a large portion of CVEs (cf. § 8.2), this figure suggests that addressing these weaknesses could substantially improve the security of update mechanisms.

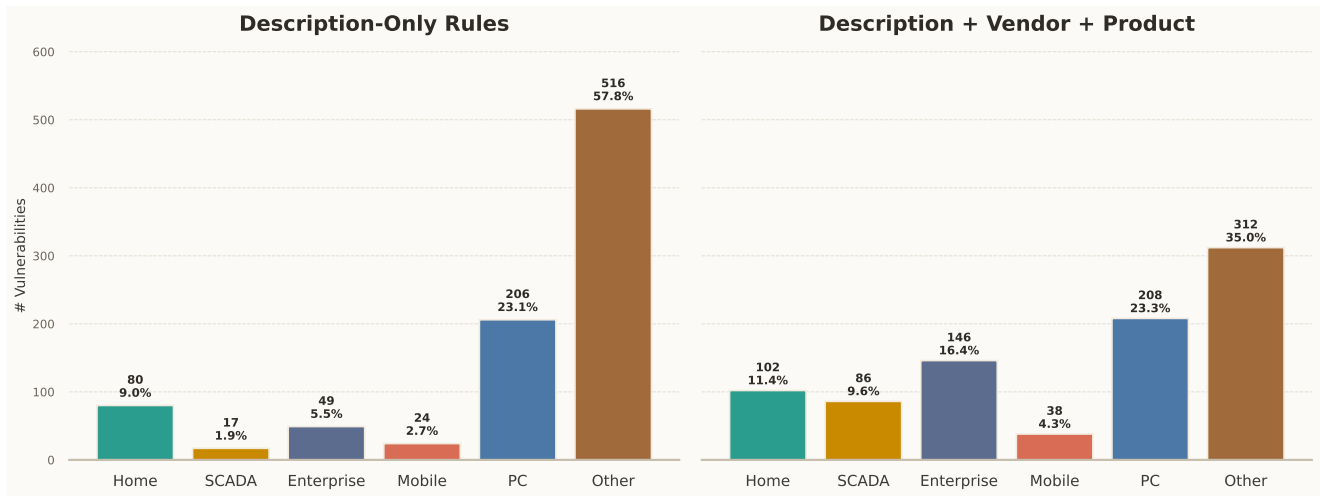


Figure 16: Target-system Identification with Rule-based Approach using Vulnerability Description, Vendor, and Product Metadata.

Based on this observation, we identify six generic mitigation recommendations that can serve as a practical checklist when designing, implementing, or deploying software updates. These recommendations are derived from two primary sources: (i) MITRE’s recommended mitigations and (ii) techniques and solutions described in the scientific literature.

MITRE provides mitigation guidance for most CWEs; however, these recommendations vary in specificity and relevance to update processes. For each of the Top 10 weaknesses, we examined the corresponding MITRE mitigations and evaluated their suitability for inclusion in the proposed update-related mitigations. The selection was guided by three criteria: (i) the mitigation must be directly relevant to update-related vulnerabilities to avoid overly generic security advice, (ii) it must be sufficiently concrete to support practical implementation, and (iii) it should have a potentially strong impact on mitigating at least one identified weakness.

The analysis in this paper resulted in four mitigation recommendations (R1-R4). In addition, we examined the scientific literature on secure software updates to identify gaps not fully addressed by MITRE’s guidance, leading to two additional recommendations (R5-R6). We do not claim that this list is exhaustive; other complementary mitigation strategies may also be relevant. Below, we present the six mitigation recommendations (R1-R6) derived from MITRE guidance and prior research.

R1: Automated Static Analysis (ASA)

ASA analyzes binary code before execution on the device. According to MITRE, ASA typically involves constructing a control-flow graph and identifying violated paths between input sources and sensitive sinks. In the context of software updates, ASA can help detect coding errors, logic flaws, and potential runtime issues in update components [38]. Automated dynamic analysis has also been shown to be effective for firmware analysis [7]. Combining

static and dynamic techniques can further improve completeness and soundness [41].

R2: Sandbox Environment

Sandboxing isolates potentially unsafe code by executing it within a restricted environment, thereby preventing malicious behavior from affecting other applications, system resources, or the host operating system. In update processes, deploying updates within a sandbox or jail environment allows the system to assess update behavior before full deployment, enforcing separation from the live environment [21]. However, the effectiveness of this mitigation depends on the sandbox’s isolation guarantees and may primarily limit the scope rather than fully prevent attacks.

R3: Input Validation

Assuming all inputs to be potentially malicious, input validation enforces strict constraints and rejects data that does not conform to expected formats or limits. In update mechanisms, this includes validating all inputs that affect raw update data and metadata, such as file formats, sizes, string lengths, and numeric parameters, to ensure compliance with device specifications.

Although conceptually straightforward, robust input validation becomes challenging when handling complex data structures. If input validation is not implemented, software developers will face even more challenges managing security update procedures [19].

R4: Environment Hardening

Environment hardening focuses on reducing the attack surface by limiting the privileges available to processes and code. In the context of updates, this involves executing update components with the minimum privileges required, such as restricting access to sensitive directories or hard-coded file paths associated with update content.

R5: Remote Attestation

Remote attestation is a security mechanism in which a trusted verifier assesses the trustworthiness of another

device (the prover) by validating its state through cryptographic evidence [25, 36]. This technique is widely used in complex, decentralized systems [17]. Of particular relevance to this work, remote attestation has been integrated into secure software update mechanisms for embedded systems [34], connected vehicles, and protections against memory-related attacks in firmware, software, and cloud environments.

R6: Update Framework

Dedicated update frameworks are designed to deliver software and firmware updates in a secure and reliable manner. A well-known example is The Update Framework (TUF), which has received extensive academic and industrial attention and can be integrated into a wide range of update systems. TUF provides resilience even when parts of the system are compromised. Uptane [22], an extension of TUF tailored for the automotive domain, further integrates update verification with remote attestation. Another example is the SWUpdate framework for Linux-based embedded systems. Overall, update frameworks offer an effective means of mitigating a broad range of update-related weaknesses.

7.2. Mapping Recommendations to Weaknesses

After defining the six mitigation recommendations, we map each recommendation to the weaknesses it addresses, resulting in a recommendation matrix. To express the potential effectiveness of each mitigation, we draw on MITRE's prevention and detection effectiveness ratings [12], which classify (EffectivenessEnumeration) **prevention effectiveness** as high, moderate, limited, defense in depth, incidental, or discouraged, and (DetectionEffectivenessEnumeration) **detection effectiveness** as high, moderate, SOAR partial, opportunistic, limited, or none.

Due to the limited availability of empirical data needed for a fine-grained assessment, we adopt a coarse-grained classification scheme. In our matrix, a mitigation is labeled as **Prevents (P)** and **Partially Prevents (PP)** when there is a clear indication of its effectiveness against a given weakness, and when only partial or limited effectiveness is supported, respectively, as a means of **prevention**. Similarly, a mitigation is labeled as **Detects (D)** and **Partially Detects (PD)** when there is a clear indication of its effectiveness against a given weakness, and when only partial or limited effectiveness is supported, respectively, as a means of **detection**. If no evidence is available to suggest effectiveness, the corresponding entry is left blank.

Table 4 presents the mapping between update-related weaknesses and the mitigation recommendations, based on their relevance within our curated dataset. A single asterisk (*) indicates that the mitigation is explicitly recommended by MITRE or the scientific literature, but without a specified effectiveness rating. A double asterisk (**) denotes that both the recommendation and its effectiveness rating are explicitly provided by the source. Below, we justify our effectiveness ratings for those mitigations, weakness pairs that are not explicitly rated by MITRE (i.e., none-double

asterisk), and the two recommendations (i.e., R5 and R6) we derived from the scientific literature.

Environment hardening (R4) can mitigate path-related weaknesses (PRW), $R4 \times PRW = (PP^*)$, as highlighted by MITRE². However, we classify this mitigation as partial prevention, since restricting privileges to specific directories or paths does not fully eliminate all attack vectors. In practice, attackers may still manipulate file paths to escape hardened directories and bypass protection mechanisms.

For example, an attacker may supply crafted path inputs such as `..%2f` or `%2e%2e/`, which both resolve to `../`, in an attempt to traverse directories or access the root filesystem. This is particularly relevant when access to local update paths does not require administrative privileges.

Remote attestation (R5) can detect malicious activities for authentication-related weaknesses (ARW): $R5 \times ARW = (D)$, as in [18, 39]. User input [31] by observing system state changes, log events, or anomalous input activity. Its effectiveness in this regard is enhanced when combined with robust input validation mechanisms (R3), leading to also mitigate input-related weaknesses (IRW), $R5 \times IRW = (D)$. It can further mitigate authorization-related weaknesses (AuzRW) $R5 \times AuzRW = (D)$, when combined with TPM 2.0 and Enhanced Authorization [25].

Remote attestation (R5) can also partially detect malicious behavior related to **path** or **directory** manipulation (PRW), $R5 \times PRW = (PD)$ [31], and the use of invalid or untrusted certificates [18] by verifying the authenticity of the device certificate against a known certified **certificate** (CRW), $R5 \times CRW = (PD)$. However, most existing attestation mechanisms are largely static and may not detect violations that occur dynamically during the update process.

Update Framework (R6) can effectively prevent authentication-related weaknesses (ARW), $R6 \times ARW = (P)$, as they are specifically designed to ensure the authenticity and integrity of update artifacts [22].

$R6 \times IRW = (PP^*)$, $R6 \times AuzRW = (PP)$, $R6 \times PRW = (PP)$: The use of a dedicated update framework can also partially prevent input-related weaknesses³ [22], especially if combined with R3. An update framework can also be used for authorization, particularly in establishing trust in IoT devices [1]. Finally, it can also partially prevent against path-related and directory weaknesses [22].

8. Discussion, Limitations, & Future Work

In this section, we discuss the results obtained from our analysis and outline the main limitations and challenges of our study. We also identify directions for future research.

8.1. Discussion

Keeping computing systems up to date remains challenging, particularly because many systems were originally developed with limited emphasis on cybersecurity and without

²($R4 \times PRW$): <https://cwe.mitre.org/data/definitions/22.html>

³($R6 \times IRW$): <https://cwe.mitre.org/data/definitions/20.html>

Table 4

Recommendation matrix for Mitigation of Update-related Weaknesses: ARW (Authentication-related CWEs), IRW (Input-related CWEs), AuzRW (Authorization-related CWEs), PRW (Path-related CWEs), CRW (Certificate-related CWEs).

Weakness	R1	R2	R3	R4	R5	R6
ARW	D**	PP**		PP**	D	P
IRW	D**		P**		D	PP*
AuzRW	D**	PP**			D	PP
PRW		PP**		PP*	PD	PP
CRW	PD**				PD	

update security as a primary design consideration. This challenge is further amplified in cyber-physical systems (CPS), where applications often operate in real-time environments that require strict timing guarantees and high availability, making updates both complex and potentially risky.

Our empirical analysis shows that a wide range of systems can be affected during the update process, including SCADA-related components such as Human–Machine Interfaces (HMI) (CVE-2021-37160), Remote Terminal Units (RTU) (CVE-2019-14926), and healthcare systems such as ApexPro Telemetry (CVE-2020-6965). Moreover, even when devices support updates, the mechanisms used to distribute and apply these updates are frequently vulnerable to exploitation.

The results further indicate that update mechanisms are often exploitable locally and exhibit relatively low barriers to exploitation. Minimal privilege requirements combined with little or no user interaction enable attackers to compromise update processes efficiently and at scale. While the scope, changed for CVSS (compared to the unchanged category), is less common in update-related vulnerabilities, any such exploitation can have severe consequences due to the inherently trusted role of update mechanisms within a system.

In addition, update-related vulnerabilities tend to have disproportionately severe impacts on confidentiality, integrity, and availability. Unlike general vulnerabilities, which span a wide range of impact levels, update-related flaws often result in high-impact consequences. These findings reinforce the need to treat update-related vulnerabilities as a distinct and high-priority class within vulnerability management and risk assessment frameworks.

The target-system approach adds an important contextual dimension to the findings of this paper. While the analyses in **RQ1** and **RQ2** characterize update-related vulnerabilities in terms of exploitability, impact, severity, and underlying weakness classes, the rule-based approach introduced in **RQ3** helps identify which kinds of systems are affected.

We show that most of the target-system classes fall into the ARW (Authentication-related CWEs). However, the most prevalent weakness in the SCADA class is CWE-823 (Use of Out-of-range Pointer Offset) as shown in Figure 15. Since this weakness does not appear among the overall top-ten update-related weaknesses, it suggests that SCADA systems

exhibit distinct vulnerability characteristics and therefore require different mitigation recommendations.

8.2. Limitations

The limited availability of CVSS metrics constrained our ability to assess the impact of both update-related and overall vulnerabilities. As this study relies on publicly available CVE data, it inherits a few limitations:

- **CVSS absence:** Out of the 892 update-related vulnerabilities, CVSS metrics are available for only 559 files, while the 242,904 overall vulnerabilities have only 147,556 files with CVSS metrics, resulting in missing 37.3% and 39.3% respectively as shown in Figure 17.
- **CWE absence:** Out of the 892 update-related vulnerabilities, cweId tag are available for only 505 files, while the 242,904 overall vulnerabilities have only 125,865 files with the cweId tag, resulting in missing 43.4% and 48.2% respectively as shown in Figure 18.
- **Erroneous classification:** Some CVEs appear to be misclassified. For example, CVE-2021-25437 is labeled as Improper Input Validation (CWE-20), although its description suggests it more closely aligns with Improper Access Control (CWE-284).
- **CWE abstraction:** Several CWEs are defined at a high level of abstraction, resulting in descriptions that are generic and applicable across many contexts.

These limitations are not unique to this study and have been reported in prior research relying on publicly disclosed vulnerability data [4, 24, 40]. We also encountered practical challenges when extracting CWE identifiers (cweId), as their location within CVE records varies (e.g., under containers.cna or containers.adp), complicating code implementation in our experiments.

The rule-based approach remains prone to error or bias toward the other class, even after using the vendor and product meta-data. For example, in CVE-2022-30262, which states that "The Emerson ControlWave 'Next Generation' RTUs through 2022-05-02 mishandle firmware integrity...", the rule-based can be further improved other class.

Despite these limitations, our findings provide guidance for prioritizing mitigation strategies for securing update mechanisms.

8.3. Future Work

The limitations discussed in this paper motivate the following directions for future research.

First, vulnerability retrieval and query generation were conducted using a semi-automated process. Although the resulting dataset was manually curated to improve accuracy and relevance, some false positives may still arise due to keyword overlap between update-related and unrelated vulnerabilities. Future work could therefore investigate fully automated retrieval and filtering pipelines that reduce manual effort while maintaining dataset quality. **Second**, the

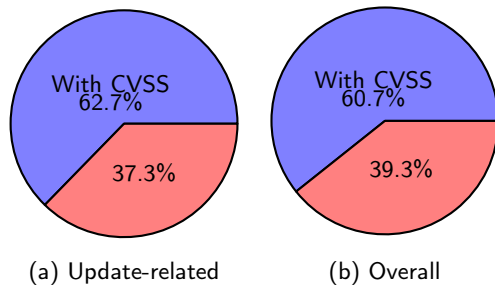


Figure 17: Missing CVSS Metrics Percentage in (in red).

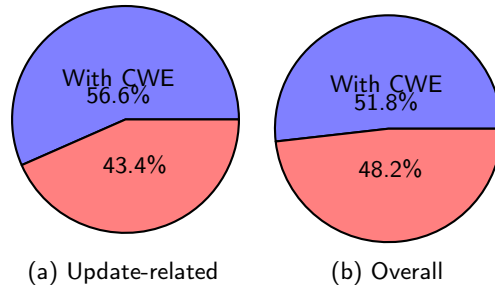


Figure 18: Missing CWE tag Percentage in (in red).

absence, incompleteness, and inconsistency of structured information in CVE records remain a challenge for large-scale automated analysis. Future work could explore more systematic methods for handling missing fields, resolving abstraction issues, and improving metadata normalization, building on the rule-based and NLP-based classification frameworks developed in this study. **Third**, update-related vulnerabilities differ in their long-term security implications: some are eventually mitigated through subsequent patches, whereas others remain unresolved because affected devices or software have reached end-of-life. A comparative analysis of these two scenarios would provide valuable insight into the lasting risk posed by update-related vulnerabilities. **Fourth**, limitations in CVSS scoring affect the assessment of impact and severity (cf. §8.2). Prior work has questioned the empirical grounding of CVSS [30], proposed alternative weighting schemes [29], and introduced new vulnerability scoring models [27, 30, 40]. Evaluating such enhanced or alternative scoring approaches in the context of update-related vulnerabilities is therefore an important direction for future research. **Finally**, future work could integrate the rule-based approach to improve the target-system identification (cf. §6), using other approaches such as natural language processing.

9. Conclusion

Software and firmware update mechanisms are fundamental to the security and reliability of modern computing systems, yet they also constitute a critical attack surface. This study has shown that the very process intended to improve security can itself introduce high risk. By analyzing a decade of disclosed vulnerabilities, we demonstrated that

update-related vulnerabilities are disproportionately severe compared with the broader vulnerability population. They more frequently result in high-impact compromises of confidentiality, integrity, and availability, and are commonly exploitable through local access with little or no user interaction and low privilege requirements. We further identified the root causes of these security weaknesses, with authentication and authorization flaws, improper input validation, and path- and certificate-related weaknesses emerging as the most common weaknesses. In addition, we introduced a rule-based approach that uses vulnerability descriptions together with vendor and product metadata to categorize vulnerable target-system classes, thereby adding deployment context to large-scale vulnerability analysis. Building on these findings, we derived six actionable recommendations, ranging from implementing robust input validation and remote attestation to adopting dedicated secure update frameworks, aimed at mitigating common weaknesses and improving the resilience of update mechanisms.

Acknowledgements

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. The second author was partly financed by the Swedish Foundation for Strategic Research (SSF), project FUS21-0033 (ASTECC).

References

- [1] Aoki, N., Takefusa, A., Ishikawa, Y., Ono, Y., Sakane, E., Aida, K.: Zt-ota update framework for iot devices toward zero trust iot. In: 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC). pp. 2195–2202 (2024). <https://doi.org/10.1109/COMPSAC61105.2024.00352>
- [2] Asokan, N., Nyman, T., Rattanavipanon, N., Sadeghi, A.R., Tsudik, G.: Assured: Architecture for secure software update of realistic embedded devices. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2018). <https://doi.org/10.1109/TCAD.2018.2858422>
- [3] Bellissimo, A., Burgess, J., Fu, K.: Secure software updates: Disappointments and new challenges. In: First USENIX Workshop on Hot Topics in Security (HotSec 06). USENIX Association (2006)
- [4] Blessing, J., Specter, M.A., Weitzner, D.J.: Cryptography in the wild: An empirical analysis of vulnerabilities in cryptographic libraries. In: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security. (ASIA CCS '24), Association for Computing Machinery (2024). <https://doi.org/10.1145/3634737.3657012>
- [5] Blinowski, G.J., Piotrowski, P.: Cve based classification of vulnerable iot systems. In: Zamojski, W.,

- Mazurkiewicz, J., Sugier, J., Walkowiak, T., Kacprzyk, J. (eds.) Theory and Applications of Dependable Computer Systems. pp. 82–93. Springer International Publishing, Cham (2020)
- [6] Bradley, C., Barrera, D.: Towards characterizing iot software update practices. In: Foundations and Practice of Security. Springer Nature (2023)
- [7] Chen, D.D., Woo, M., Brumley, D., Egele, M.: Towards automated dynamic analysis for linux-based embedded firmware. In: (NDSS) (2016)
- [8] Coppens, B., De Sutter, B., De Bosschere, K.: Protecting your software updates. IEEE Security and Privacy **11**(2), 47–54 (Mar 2013). <https://doi.org/10.1109/MSP.2012.113>
- [9] Costin, A., Zaddach, J., Francillon, A., Balzarotti, D.: A large-scale analysis of the security of embedded firmwares. In: 23rd USENIX security symposium (USENIX Security 14) (2014)
- [10] CVE Program: Common vulnerabilities and exposures (cve). <https://www.cve.org>, accessed: 2026-01-03
- [11] CVE Program: Cve list v5 repository. <https://github.com/CVEProject/cvelistV5>, accessed: 2026-01-03
- [12] CWE Program: Cwe schema documentation. <https://cwe.mitre.org/documents/schema/index.html>, accessed: 2026-01-03
- [13] CWE Program: Cwe top 25 most dangerous software weaknesses. https://cwe.mitre.org/top25/archive/2023/2023_top25_list.html, accessed: 2026-01-03
- [14] FIRST: Common Vulnerability Scoring System v3.1: Specification Document, <https://www.first.org/cvss/v3-1/specification-document>, accessed: 2025-01-06
- [15] Froh, F.N., Gobbi, M.F., Kinder, J.: Differential static analysis for detecting malicious updates to open source packages. In: Proceedings of the 2023 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses. p. 41–49. SCORED '23, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3605770.3625211>
- [16] Ibrahim, M., Continella, A., Bianchi, A.: Aot - attack on things: A security analysis of iot firmware updates. In: 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P) (2023). <https://doi.org/10.1109/EuroSP57164.2023.00065>
- [17] Khodari, M., Rawat, A., Asplund, M., Gurtov, A.: Decentralized firmware attestation for in-vehicle networks. In: Proceedings of the 5th on Cyber-Physical System Security Workshop. CPSS '19, Association for Computing Machinery (2019). <https://doi.org/10.1145/3327961.3329529>
- [18] Khurshid, A., Raza, S.: Autocert: Automated toctou-secure digital certification for iot with combined authentication and assurance. Computers & Security (2023). <https://doi.org/https://doi.org/10.1016/j.cose.2022.102952>
- [19] Kjøien, G.M., Øverlier, L.: A call for mandatory input validation and fuzz testing. Wireless Personal Communications (2023). <https://doi.org/10.1007/s11277-023-10431-2>
- [20] Li, F., Paxson, V.: A large-scale empirical study of security patches. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. (CCS '17), Association for Computing Machinery (2017). <https://doi.org/10.1145/3133956.3134072>
- [21] Liu, S., Feng, P., Wang, S., Sun, K., Cao, J.: Enhancing malware analysis sandboxes with emulated user behavior. Computers & Security (2022). <https://doi.org/https://doi.org/10.1016/j.cose.2022.102613>
- [22] Lorch, R., Larraz, D., Tinelli, C., Chowdhury, O.: A comprehensive, automated security analysis of the uptane automotive over-the-air update framework. In: Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses. (RAID '24), Association for Computing Machinery (2024). <https://doi.org/10.1145/3678890.3678927>
- [23] Nikbakht Bideh, P., Gehrmann, C.: Rosym: Robust symmetric key based iot software upgrade over-the-air. In: Proceedings of the 4th Workshop on CPS & IoT Security and Privacy. (CPSIoTSec '22), Association for Computing Machinery (2022). <https://doi.org/10.1145/3560826.3563381>
- [24] Plaka, R., Asplund, M., Nadjm-Tehrani, S.: Vulnerability analysis of an electric vehicle charging ecosystem. In: Critical Information Infrastructures Security. Springer Nature Switzerland (2024)
- [25] Plappert, C., Fuchs, A.: Secure and lightweight ecu attestations for resilient over-the-air updates in connected vehicles. In: Proceedings of the 39th Annual Computer Security Applications Conference. p. 283–297. ACSAC '23, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3627106.3627202>
- [26] Prakash, V., Xie, S., Huang, D.Y.: Inferring software update practices on smart home iot devices through user agent analysis. In: Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses. (SCORED'22), Association for Computing Machinery (2022). <https://doi.org/10.1145/3560835.3564551>

- [27] Seker, E., Meng, W.: Xvrs: Extended vulnerability risk scoring based on threat intelligence. In: 2023 IEEE International Conference on Metaverse Computing, Networking and Applications (MetaCom) (2023). <https://doi.org/10.1109/MetaCom57706.2023.00094>
- [28] Setayeshfar, O., Rhee, J.J., Kim, C.H., Lee, K.H.: Find my sloths: Automated comparative analysis of how real enterprise computers keep up with the software update races. In: Detection of Intrusions and Malware, and Vulnerability Assessment. Springer International Publishing (2021)
- [29] Spanos, G., Sioziou, A., Angelis, L.: Wivss: a new methodology for scoring information systems vulnerabilities. In: Proceedings of the 17th Panhellenic Conference on Informatics. (PCI '13), Association for Computing Machinery (2013). <https://doi.org/10.1145/2491845.2491871>
- [30] Spring, J., Hatleback, E., Householder, A., Manion, A., Shick, D.: Time to change the cvss? IEEE Security & Privacy (2021). <https://doi.org/10.1109/MSEC.2020.3044475>
- [31] Toffalini, F., Losiouk, E., Biondo, A., Zhou, J., Conti, M.: ScaRR: Scalable runtime remote attestation for complex systems. In: 22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019). USENIX Association, Chaoyang District, Beijing (2019), <https://www.usenix.org/conference/raid2019/presentation/toffalini>
- [32] Treetippayaruk, S., Senivongse, T.: Security vulnerability assessment for software version upgrade. In: 2017 18th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD) (2017). <https://doi.org/10.1109/SNPD.2017.8022734>
- [33] Usman, A.B.: On the inconsistency of update-related vulnerabilities. In: Furnell, S., Clarke, N. (eds.) Human Aspects of Information Security and Assurance. pp. 420–434. Springer Nature Switzerland, Cham (2026)
- [34] Usman, A.B., Asplund, M.: Remote attestation with software updates in embedded systems. In: 2024 IEEE Conference on Communications and Network Security (CNS) (2024). <https://doi.org/10.1109/CNS62487.2024.10735526>
- [35] Usman, A.B., Asplund, M.: Update at your own risk: Analysis and recommendations for update-related vulnerabilities. In: Nemec Zlatolas, L., Rannenber, K., Welzer, T., Garcia-Alfaro, J. (eds.) ICT Systems Security and Privacy Protection. pp. 97–110. Springer Nature Switzerland, Cham (2025)
- [36] Usman, A.B., Cole, N., Asplund, M., Boeira, F., Vestlund, C.: Remote attestation assurance arguments for trusted execution environments. In: Proceedings of the 2023 ACM Workshop on Secure and Trustworthy Cyber-Physical Systems. (SaT-CPS '23), Association for Computing Machinery (2023). <https://doi.org/10.1145/3579988.3585056>
- [37] Villegas, M.M., Orellana, C., Astudillo, H.: A study of over-the-air (ota) update systems for cps and iot operating systems. In: Proceedings of the 13th European Conference on Software Architecture. ECSA '19, Association for Computing Machinery (2019). <https://doi.org/10.1145/3344948.3344972>
- [38] Wedyan, F., Alrmuny, D., Bieman, J.M.: The effectiveness of automated static analysis tools for fault detection and refactoring prediction. In: 2009 International Conference on Software Testing Verification and Validation (2009). <https://doi.org/10.1109/ICST.2009.21>
- [39] Wilson, J., Asplund, M., Johansson, N., Boeira, F.: Provably secure communication protocols for remote attestation. In: Proceedings of the 19th International Conference on Availability, Reliability and Security. (ARES '24), Association for Computing Machinery (2024). <https://doi.org/10.1145/3664476.3664485>
- [40] Wu, Q., Xiao, Y., Liao, X., Lu, K.: OS-Aware vulnerability prioritization via differential severity analysis. In: 31st USENIX Security Symposium (USENIX Security 22). USENIX Association (2022)
- [41] Wu, Y., Wang, J., Wang, Y., Zhai, S., Li, Z., He, Y., Sun, K., Li, Q., Zhang, N.: Your firmware has arrived: A study of firmware update vulnerabilities. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association (2024)

Declaration of interests

☐The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☒The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Ahmad B. Usman reports financial support was provided by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. Mikael Asplund reports financial support was provided by the Swedish Foundation for Strategic Research (SSF). If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.